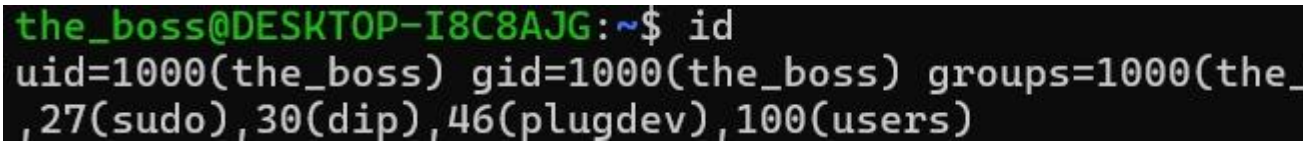
WEEK-1 PRACTICAL

1. Id:



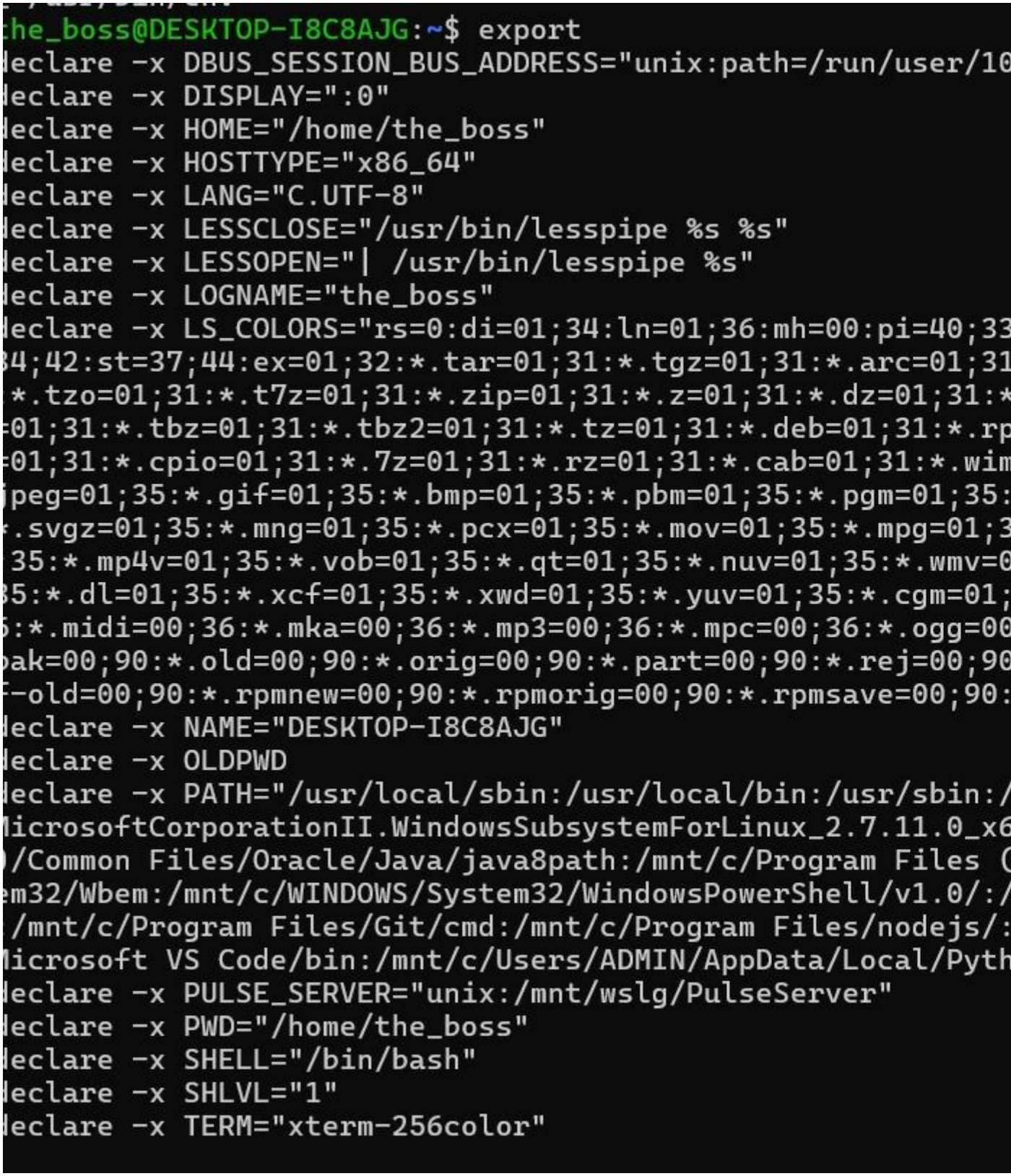
   Report: I used the `id` command to display information about the current user. It showed my user ID, group ID and the groups associated with my user account.

2. Env:

```
the_boss@DESKTOP-I8C8AJG:~$ env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=DESKTOP-I8C8AJG
PWD=/home/the_boss
LOGNAME=the_boss
HOME=/home/the_boss
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/1934_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35
44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01
:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31
=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*
o=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.
.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;
:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=
1;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.as
5:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=
36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=
old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.swp=00
*.rpmnew=00;90:*.rpmorig=00;90:*.rpmsave=00;90:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=the_boss
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/
porationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb
es/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Comm
nt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WIN
ram Files/Git/cmd:/mnt/c/Program Files/nodejs/:/mnt/c/Us
 Code/bin:/mnt/c/Users/ADMIN/AppData/Local/Python/bin:/m
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
```

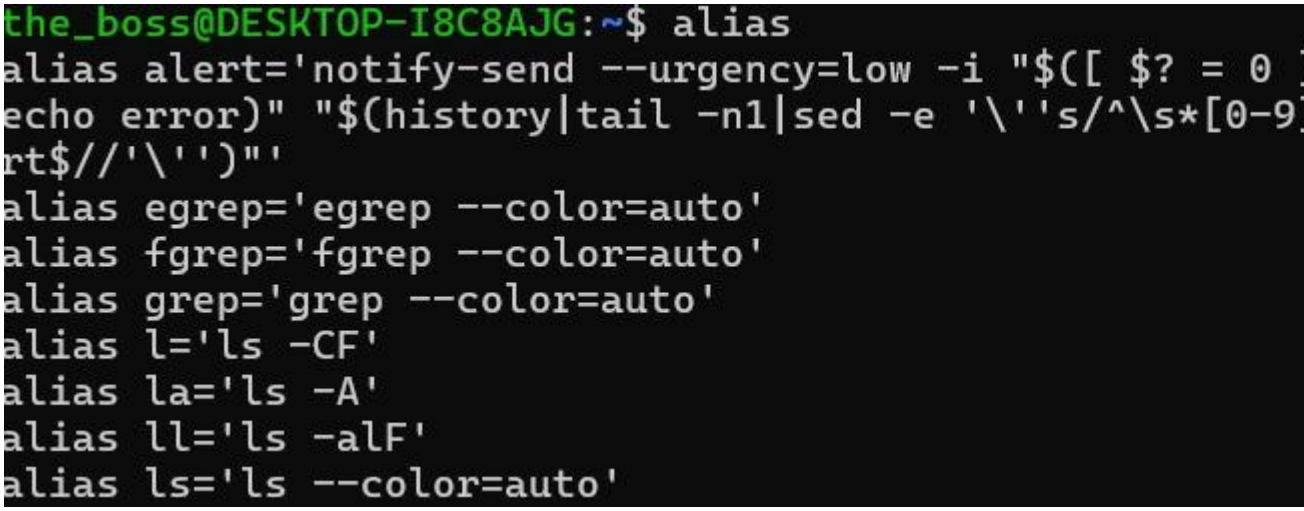
Report: I used the `env` command to display the environment variables available in my current shell. It helped me understand the different settings and variables used by the system.

3.export:



Report: I used the `export` command to create or set an environment variable. The variable can then be accessed by commands and programs running from the current shell.

3. Alias:



Report: I used the `alias` command to create a short name for a command. It helps me execute frequently used commands more easily without typing the complete command every time.

Task 1: Develop a C program that

demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID

CODE:

pavani@DESKTOP-38OLEIN: ~

GNU nano 8.7.1 command_executor.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        execlp(command, command, NULL);

        printf("Command execution failed!\n");
    }

    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-38OLEIN: ~
pavani@DESKTOP-38OLEIN:~$ nano command_executor.c
pavani@DESKTOP-38OLEIN:~$ gcc command_executor.c -o command_executor
pavani@DESKTOP-38OLEIN:~$ ./command_executor
Enter a Linux command: ls

Parent Process
Parent PID : 546
Child PID  : 547

Child Process
Child PID : 547
Parent PID: 546

Executing command: ls

OSSP   command_executor    file2.txt  hello.sh        os-lab     shortcut  txt
a.txt  command_executor.c  hello      helloCopy.java  ossp       snap
b.txt  file1.txt           hello.c    main.java       script.sh  test.txt

Child process completed.
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using for The uname -a command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.k(), and executed the command using the exec() system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

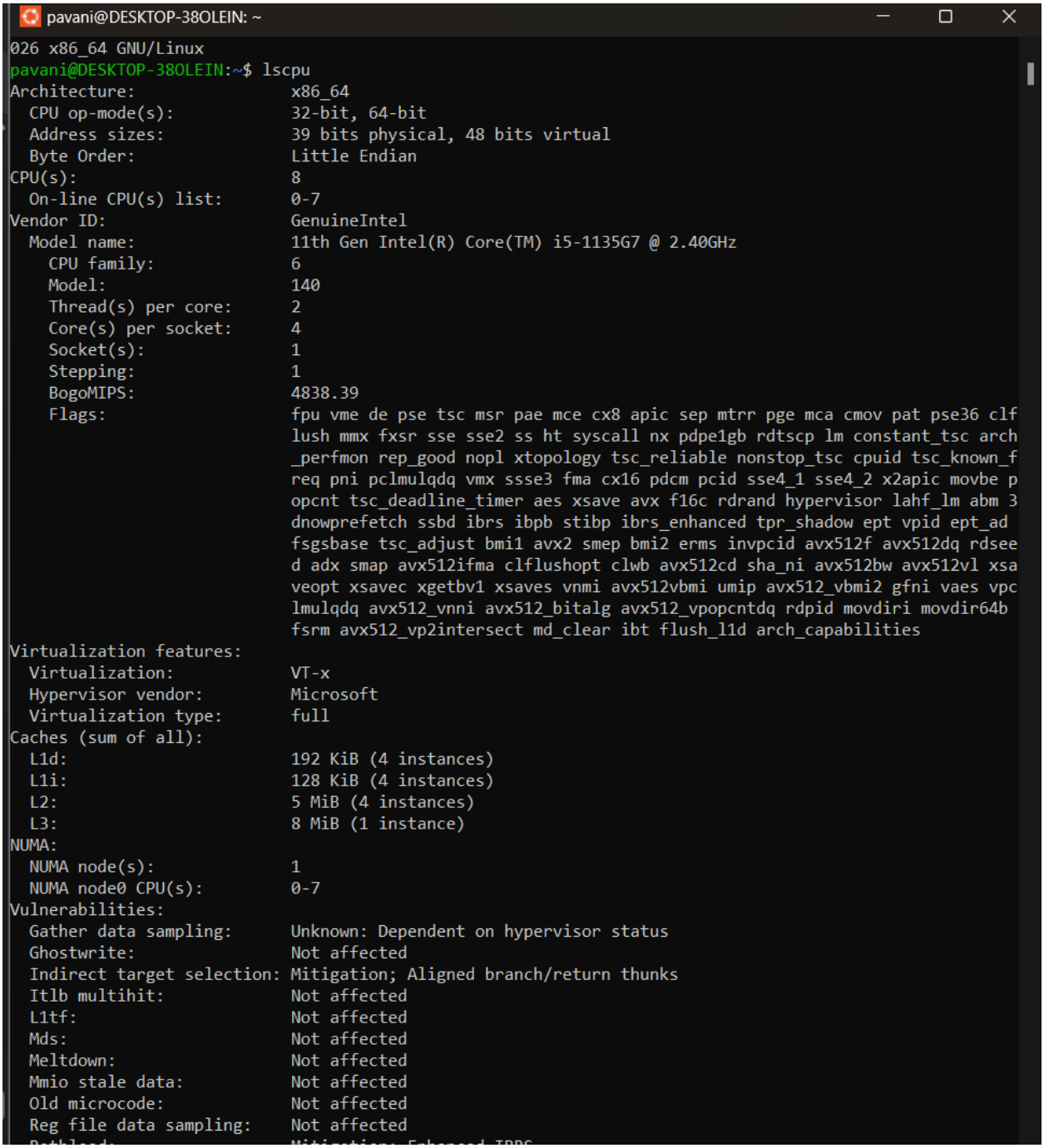
Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a :

```
pavani@DESKTOP-38OLEIN:~$ uname -a
Linux DESKTOP-38OLEIN 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2
026 x86_64 GNU/Linux
```

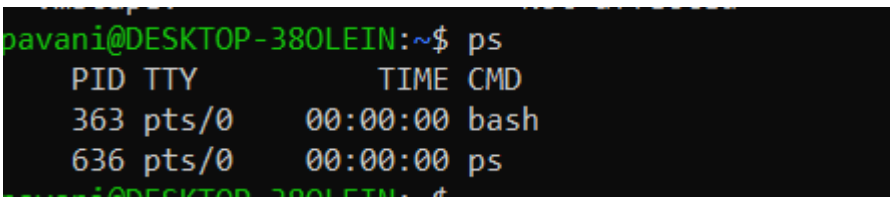
REPORT: The uname -a command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:



REPORT: The lscpu command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

Ps:



REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:

```
pavani@DESKTOP-38OLEIN:~$ top
top - 09:05:15 up 7 min,  1 user,  load average: 0.05, 0.05, 0.01
Tasks:  28 total,   1 running,  27 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3803.6 total,   2402.1 free,    555.3 used,    982.8 buff/cache
MiB Swap:   1024.0 total,   1024.0 free,      0.0 used.   3248.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   24384  15472  11616 S   0.0   0.4   0:00.76 systemd
      2 root      20   0    3180   2208   2072 S   0.0   0.1   0:00.01 init-systemd(Ub
      6 root      20   0    3180   2040   1940 S   0.0   0.1   0:00.00 init
     46 root      19  -1   50396  16892  15592 S   0.0   0.4   0:00.21 systemd-journal
     77 systemd+  20   0   22540  14656  12032 S   0.0   0.4   0:00.07 systemd-resolve
     82 root      20   0   35560  12588   9316 S   0.0   0.3   0:00.19 systemd-udevd
    144 root      20   0  159348   1836   1508 S   0.0   0.0   0:00.00 snapfuse
    150 root      20   0  454144  11612   1556 S   0.0   0.3   0:02.29 snapfuse
    177 root      20   0    2888   2016   1884 S   0.0   0.1   0:00.05 chronyd-starter
    178 root      20   0    4472   3168   2924 S   0.0   0.1   0:00.01 cron
    179 message+  20   0    8816   5508   4824 S   0.0   0.1   0:00.05 dbus-daemon
    183 root      20   0   44092  29820  14676 S   0.0   0.8   0:00.49 networkd-dispat
    186 root      20   0 2220748  38668  25020 S   0.0   1.0   0:00.85 snapd
    187 root      20   0   18444   9544   8292 S   0.0   0.2   0:00.07 systemd-logind
    190 root      20   0 1857652  15260  12616 S   0.0   0.4   0:00.15 wsl-pro-service
    207 syslog    20   0  220548   6020   4668 S   0.0   0.2   0:00.06 rsyslogd
    210 root      20   0    5492   2828   2644 S   0.0   0.1   0:00.01 agetty
    222 root      20   0  123456  32588  17976 S   0.0   0.8   0:00.41 unattended-upgr
    228 _chrony   20   0   24252  11904   7304 S   0.0   0.3   0:00.10 chronyd
    249 _chrony   20   0   12092   2512   1868 S   0.0   0.1   0:00.01 chronyd
    361 root      20   0    3188   1112    980 S   0.0   0.0   0:00.00 SessionLeader
    362 root      20   0    3204   1248   1108 S   0.0   0.0   0:00.01 Relay(363)
    363 pavani    20   0    6268   5600   3928 S   0.0   0.1   0:00.06 bash
    364 root      20   0    8648   5576   4732 S   0.0   0.1   0:00.01 login
    432 pavani    20   0   22340  13656  11032 S   0.0   0.4   0:00.10 systemd
    434 pavani    20   0   22912   4080   2076 S   0.0   0.1   0:00.00 (sd-pam)
    463 pavani    20   0    6136   5548   3992 S   0.0   0.1   0:00.02 bash
    646 pavani    20   0    8196   6084   3960 R   0.0   0.2   0:00.02 top
```

REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.